

main

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import random
4
5  from scipy.stats import norm
6
7  from Converter_dB import convert_SNR_dB_to_SNR, convert_SNR_to_SNR_dB
8  from AWGN_generator import add_AWGN_into_sequence
9  from Plotting_constellation import plot_constellation
10 from BPSK_mapper import BPSK_mapper
11 from BPSK_receiver import BPSK_receiver
12
13
14 PI = np.pi
15
16 ## task addendum, not official
17 # An error may occur here due to the way the function=demodulate_symbols_in_t
ime operates.
18 # For large values of bit_num, an error accumulates when the signal
19 # is divided into symbol periods and assigned to specific symbols.
20 # As a result, part of the bits is lost during demodulation.
21 # To avoid this, increase the bit_rate, or reduce the subdivision of
22 # the time variable t, but not excessively, since the function may
23 # incorrectly compute the intervals. Alternatively, to avoid this issue,
24 # use a fixed number of samples by rewriting the demodulation function.
25 ##
26
27 ## Model parameters
28 Energy_of_impulse = 1
29
30
31 bit_rate = 1 # need correct setting
32 numb_bits = int(40000)
33
34
35
36 # snrs_dB
37 SNRs_dB = np.arange(10, 11, 0.25)
38
39 # result BERs
40 BERs = np.zeros(np.size(SNRs_dB))
41 BERs_teor= np.zeros(np.size(SNRs_dB))
42
43
44 ## Create objects
45 bpsk_mapper = BPSK_mapper(bit_rate, Energy_of_impulse)
46 bpsk_demod = BPSK_receiver(bit_rate, Energy_of_impulse, np.array([0.5, 0.5]))
47
48
```

```

49  ## Start performing model of receiving
50  for num_ex in range(0, np.size(SNRs_dB)):
51      cur_SNR_dB = SNRs_dB[num_ex]
52
53      # Generate bit sequence
54      bit_seq_tx = [random.randint(0, 1) for _ in range(num_bits)]
55
56      # Set correct time division
57      T_symb = bpsk_mapper._T_symb
58      dt = T_symb/3 + 1e-6
59      t_start = 0
60      t_end = t_start + np.size(bit_seq_tx)/bit_rate - dt
61      t = np.arange(t_start, t_end, dt)
62
63      # Mapping
64      baseband_signal = bpsk_mapper.get_baseband_signal(bit_seq_tx)
65      baseband_signals_out = baseband_signal(t)
66
67      # add AWGN
68      baseband_signal_with_noise = add_AWGN_into_sequence((baseband_signals_out
t), cur_SNR_dB)
69
70      # demodulate
71      bit_seq_rx = bpsk_demod.demodulate_symbols_in_time(t, baseband_signal_wit
h_noise, cur_SNR_dB)
72
73      # plotting constellation
74      if cur_SNR_dB == 10:
75          plot_constellation(baseband_signals_out, baseband_signal_with_noise,
"Constellation for SNR_dB=" + str(cur_SNR_dB))
76
77      # solving BER
78      num_errors = np.sum(bit_seq_tx != bit_seq_rx) # there is may be an error
79      BERs[num_ex] = num_errors / np.size(bit_seq_tx)
80
81      # Theoretical value of error's probability
82      BERs_teor[num_ex] = norm.sf(np.sqrt(2*convert_SNR_dB_to_SNR(cur_SNR_dB)))
83
84      print(cur_SNR_dB)
85
86
87
88  ## Plotting results BER curves
89  fig5, axs = plt.subplots(1, 1, figsize=(10, 8))
90
91  axs.semilogy(SNRs_dB, BERs, '.', label='Practical error\'s probability')
92  axs.semilogy(SNRs_dB, BERs_teor, '--', label='Teoretical error\'s probabilit
y')
93  axs.set_title('curves of BER')
94  axs.legend()
95  axs.set_xlabel('SNR dB')
96  axs.set_ylabel('BER')

```

```
97  axs.grid(True)
98
99  plt.tight_layout()
100 plt.show()
101
102
103
104
```